

POTHIREDDY.BHAVYA SRI

CSE-A

24AG1A0546

08-09-2025:

Monday;

◆Java Notes:Arrays

Declaration of array:

Arrays:

- Arrays are defined as collections of similar types of data items stored at contiguous memory locations.
- Array is the simplest data structure where each data element can be randomly accessed by using its index number.

Array declaration:

- Java: `int arr[10]; char arr[10]; float arr[5];`

What is an array?

In Java and many other languages, arrays are **static(fixed size)**. Array organizes items sequentially, one after another, in memory.

- The items could be Integer, String, Object, – anything. The items are stored in **contiguous** (adjacent to each other) memory locations.
- Each position in the array has an index, starting at the 0th index. In Java, integers take 4 bytes, so the memory addresses of each adjacent element are added by 4 bytes.

- If we say our array memory, location/address starts from 100, then the following integer address will start from 104(100+4) bytes, and so on.

Declaration and initialization

- Consider an array `A[]` that has 5 elements. To access the last element of the array, we use `A[4]`.
- With this knowledge, if `N` is the array length, then `(N-1)` is how we access the last element.
- There are two ways we can declare and initialize the array in Java.

Example:

```
int[] arr = new int[5];
```

```
// stores 5 items, capacity = 5 and size is 0
```

```
// (no items added, so far)
```

```
System.out.println(Arrays.toString(arr));
```

```
// [0, 0, 0, 0, 0]
```

*)Initially, we did not add any items to the array, so the array values are defaulted to 0 as seen above.

There are two approaches to initialize the array

// approach 1:

```
int[] A = new int[5];
```

A[0] = 1;

A[1] = 2;

A[2] = 3;

A[3] = 4;

A[4] = 5;

// approach 2:

int[] A = {1, 2, 3, 4, 5};

Accessing array elements:

- Since arrays in Java start from 0 th index. If you want to access the first element, you need to give A[0], and A[1] for accessing the second element, and so on A[N-1] to access the last element.
- What happens if we do A[-100], A[N], and A[N+1]? You will get **ArrayIndexOutOfBoundsException**
- Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 100 out of bounds for length 5 at array.ArrayIntroduction.main(Example.java:20)

EXAMPLE SUMS OF BASIC ARRAY:

```
import java.util.Scanner;

public class sumarray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("enter no.of elements");
```

```

    int n = sc.nextInt();

    int[] arr = new int[n];

    System.out.print("enter  elements");

    for (int i = 0; i < n; i++) {

        arr[i] = sc.nextInt();

    }

    int sum = 0;

    for (int i = 0; i < n; i++) {

        sum += arr[i];

    }

    System.out.print(sum);

    sc.close();

}
}

```

SUM OF EVEN ELEMENTS OF ARRAY:

```

import java.util.Scanner;

public class evenarraysum {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("enter no.of elements");

        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.print("enter  elements: ");

        for (int i = 0; i < n; i++) {

            arr[i] = sc.nextInt();

        }

    }

}

```

```

int sum = 0;

for (int i = 0; i < n; i++) {

    if (i % 2 == 0) { // if even index positioned sum then instead if i%2==0 it is arr[i]%2==0

        sum += arr[i];

    }

}

System.out.print(sum);

sc.close();

}

```

✓ Functions of Array:

1) arr.length():

```

public class arrlength {

    public static void main(String[] args) {

        int[] arr = { 5, 8, 3, 4, 2 };

        System.out.print(arr.length);

    }

}

```

Op: 5

2) sort():

```

import java.util.Arrays;

public class arrlength {

    public static void main(String[] args) throws java.lang.Exception {

        int[] a = { 5, 8, 3, 4, 2 };

        Arrays.sort(a);

        System.out.println(Arrays.toString(a));

    }

}

```

Op:

[2, 3, 4, 5, 8]

Example2:

```

import java.util.Arrays;

public class arrlength {

    public static void main(String[] args) throws java.lang.Exception {

        // your code goes here
    }

}

```

```

int[] a1 = { 5, 8, 3, 4, 2 };
int[] a2 = a1;
Arrays.sort(a1);
System.out.println(Arrays.toString(a1));
System.out.println(Arrays.toString(a2));
}
}

```

Op:

[2, 3, 4, 5, 8]

[2, 3, 4, 5, 8]

Example3:

```

import java.util.Arrays;
public class arlength {
    public static void main(String[] args) throws java.lang.Exception {
        // your code goes here
        int[] a1 = { 5, 8, 3, 4, 2 };
        int[] a2 = a1;
        a1[0] = 100;
        System.out.println(Arrays.toString(a1));

        System.out.println(Arrays.toString(a2));    }
}

```

Op:

op:

[100, 8, 3, 4, 2]

[100, 8, 3, 4, 2]

=== Code Execution Successful ===

Example4:

```

import java.util.Arrays;
public class arlength {
    public static void main(String[] args) throws java.lang.Exception {
        // your code goes here
        int[] a1 = { 5, 8, 3, 4, 2 };
        int[] a2 = Arrays.copyOf(a1, a1.length);
        a1[0] = 100;
        System.out.println(Arrays.toString(a1));
        System.out.println(Arrays.toString(a2));
    }
}

```

Op:

[2, 3, 4, 5, 8]

=== Code Execution Successful ===

Example5:

```

import java.util.Arrays;
public class arlength {

```

```
public static void main(String[] args) throws java.lang.Exception {
    // your code goes here
    int[] a1 = { 5, 8, 3, 4, 2 };
    int[] a2 = Arrays.copyOfRange(a1, 1, 4);

    System.out.println(Arrays.toString(a1));
    System.out.println(Arrays.toString(a2)); } }
```

op:

[5, 8, 3, 4, 2]

[8, 3, 4]

Example 6:

```
import java.util.Arrays;
public class arrlength {
    public static void main(String[] args) {
        int[] arr = new int[5];
        System.out.println(Arrays.toString(arr));
    }
}
```

Op:

[0, 0, 0, 0, 0]

3) fill():

Example1:

```
import java.util.Arrays;
public class arrlength {
    public static void main(String[] args) {
        int[] arr = new int[15];
        Arrays.fill(arr, -1);
        System.out.println(Arrays.toString(arr)); } }
```

op:

[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]

SORT ARRAY IN DESCENDING ORDER:

```
import java.util.Arrays;

public class arraydecendingsort {
    public static void main(String[] args) {

        int[] arr = { 5, 7, 2, 3, 1, 4 };
        int n = arr.length;

        Arrays.sort(arr);
        for (int i = 0; i < n / 2; i++) {
            int t = arr[i];
```

```

        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = t;
    }

    System.out.print(Arrays.toString(arr));
}
}

```

Find array element:

```

import java.util.Arrays;

public class searchelementarra {
    public static void main(String[] args) {
        int[] arr = { 2, 3, 4, 7, 8, 10, 11 };
        int x = 8;
        System.out.print(Arrays.binarySearch(arr, x));
    }
}

```

Op:

4

AVERAGE OF ELEMENTS :

```

import java.util.Scanner;

public class averagearray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int sum = 0;
        int avg = 0;
        for (int i = 0; i < n; i++) {
            sum += arr[i];
        }
        avg = (sum) / n;
        System.out.print(avg);
        sc.close();
    }
}

```

Op:

enter no.of elements6

enter elements: 3 6 4 6 9 2

5

COUNT ODD ELEMENTS OF ARRAY:

```
import java.util.Scanner;

public class countoddarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int count = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] % 2 != 0)
                count++;
        }
        System.out.print(count + " ");
        sc.close();
    }
}
```

Op:

enter no.of elements6

enter elements: 3 5 3 6 8 2

3

SUM OF ODD ELEMENTS OF ARRAY:

```
import java.util.Scanner;

public class oddsumarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int sum = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] % 2 != 0)
                sum += arr[i];
        }
    }
}
```

```

    }
    System.out.print(sum);
    sc.close();
}
}

```

enter no.of elements6

enter elements: 3 5 6 9 2 4

17

COUNT OF ELEMENTS THAT ARE GREATER THAN EACH ELEMENT IN ARRAY:

```

import java.util.Scanner;

public class greatercountofelements {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }
        for (int j = 0; j < n; j++) {
            int m = 0;
            for (int k = 0; k < n; k++) {
                if (a[j] < a[k]) {
                    m++;
                }
            }
            System.out.print(m + " ");
        }
        sc.close();
    }
}

```

Op:

6

3 5 7 8 2 4

4 2 1 0 5 3

PRINT YES IF ELEMENT HAS GREATER VALUE THAN IT IN THE ARRAY:

```

import java.util.Scanner;

public class graterthanelementsyesorno {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
    }
}

```

```

for (int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}
int max = arr[0];
for (int i = 1; i < n; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}
for (int i = 0; i < n; i++) {
    if (arr[i] < max) {
        System.out.print("Yes ");
    } else {
        System.out.print("No ");
    }
}
sc.close();
}
}

```

Op:

7

5 9 4 7 2 4 1

YesNoYesYesYesYesYes

DOUBLE EACH ELEMENT IN ARRAY:

```

import java.util.Scanner;

public class doubletheelementarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        for (int i = 0; i < n; i++) {
            arr[i] = arr[i] * 2;
            System.out.print(arr[i] + " ");
        }

        sc.close();
    }
}

```

OP:

enter no.of elements6

enter elements: 4 6 9 2 1 7
8 12 18 4 2 14

REVERSE THE ELEMENTS OF ARRAY:

```
import java.util.Scanner;

public class reverseofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        int[] rev = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        for (int i = 0; i < n; i++) {
            rev[i] = arr[n - 1 - i];
        }

        System.out.print("Reversed Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(" " + rev[i]);
        }
        System.out.println();
        sc.close();
    }
}
```

Op:

enter no.of elements6
enter elements: 3 6 8 9 1 2
Reversed Array: 2 1 9 8 6 3

ASCENDING ORDER OF ARRAY:

```
import java.util.*;

public class ascendingorderarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter elements of array:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }
}
```

```

int count = 0;
for (int i = 0; i < n - 1; i++) {
    if (arr[i] <= arr[i + 1]) {
        count++;
    }
}

if (count == n - 1) {
    System.out.print("Array is in ascending order.");
} else {
    System.out.print("Array is NOT in ascending order.");
}
sc.close();
}
}

```

Op:

Enter size of array: 6

Enter elements of array:

3 5 8 4 1 9

Array is NOT in ascending order.

MAKE NEGATIVE ELEMENTS ZERO IN OUTPUT:

```

import java.util.Scanner;

public class negativetozero {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        for (int i = 0; i < n; i++) {
            if (arr[i] < 0)
                arr[i] = 0;
            System.out.print(arr[i] + " ");
        }

        sc.close();
    }
}

```

enter no.of elements6

enter elements: 3 -4 6 -2 1 7

-----3 0 6 0 1 7

FREQUENCY OF ELEMENTS OF ARRAY:

```
import java.util.Scanner;
public class frequencyofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        int[] freq = new int[n];
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            freq[i] = -1;
        }
        for (int i = 0; i < n; i++) {
            int count = 1;
            if (freq[i] == 0)
                continue;

            for (int j = i + 1; j < n; j++) {
                if (arr[i] == arr[j]) {
                    count++;
                    freq[j] = 0;
                }
            }
            freq[i] = count;
        }

        System.out.println("Element | Frequency");
        for (int i = 0; i < n; i++) {
            if (freq[i] != 0) {
                System.out.println(" " + arr[i] + " -- " + freq[i]);
            }
        }

        sc.close();
    }
}
```

Op:

Enter size of array: 7

Enter 7 elements:

3 6 3 2 1 1 3

Element | Frequency

3 -- 3

6 -- 1

2 -- 1

1 -- 2

MAXIMUM ELEMENT OF ARRAY:

```
import java.util.Scanner;

public class maxofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int max = arr[0];
        for (int i = 0; i < n; i++) {
            if (arr[i] > max)
                max = arr[i];
        }
        System.out.print(max);
        sc.close();
    }
}
```

Op:

enter no.of elements6

enter elements: 5 3 7 9 2 1

9

MINIMUM ELEMENT OF ARRAY:

```
import java.util.Scanner;

public class minofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int min = arr[0];
        for (int i = 0; i < n; i++) {
```

```

        if (arr[i] < min)
            min = arr[i];
    }
    System.out.print(min);
    sc.close();
}
}

```

Op:

enter no.of elements6

enter elements:6 3 6 8 1 2

SECOND LARGEST ELEMENT IN ARRAY:

```

import java.util.Scanner;

public class secondlargestofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] a = new int[n];
        System.out.print("enter elements");
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }
        int largest, secondlargest;
        if (a[0] > a[1]) {
            largest = a[0];
            secondlargest = a[1];
        } else {
            largest = a[1];
            secondlargest = a[0];
        }
        for (int i = 2; i < n; i++) {
            if (a[i] > largest) {
                secondlargest = largest;
                largest = a[i];
            } else if (a[i] > secondlargest && a[i] != largest) {
                secondlargest = a[i];
            }
        }

        System.out.println("Largest element = " + largest);
        System.out.println("Second largest element = " + secondlargest);
        sc.close();
    }
}

```


}

Op:

enter no.of elements7

enter elements4 7 8 3 2 9 1

Largest element = 9

Second largest element = 8

SECOND SMALLEST ELEMENT OF ARRAY:

```

import java.util.Scanner;
public class secondsmallestofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements: ");
        int n = sc.nextInt();
        int[] a = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }
        int smallest, secondsmallest;
        if (a[0] > a[1]) {
            smallest = a[0];
            secondsmallest = a[1];
        } else {
            smallest = a[1];
            secondsmallest = a[0];
        }
        for (int i = 2; i < n; i++) {
            if (a[i] < smallest) {
                secondsmallest = smallest;
                smallest = a[i];
            } else if (a[i] < secondsmallest && a[i] != smallest) {
                secondsmallest = a[i];
            }
        }
        System.out.println("smallest element = " + smallest);
        System.out.println("Second smallest element = " + secondsmallest);
        sc.close();
    }
}

```

Op:

enter no.of elements: 6

enter elements: 3 6 8 4 2 9

smallest element = 2

Second smallest element = 4

SUM OF REVERSED ARRAY WITH ORIGINAL ARRAY WITH INDEX:

```
import java.util.Scanner;

public class sumofreversearray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        int[] rev = new int[n];
        int[] sum = new int[n];
        System.out.print("enter elements");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        for (int i = 0; i < n; i++) {
            rev[i] = arr[n - 1 - i];
        }
        for (int i = 0; i < n; i++) {
            sum[i] = arr[i] + rev[i];
        }
        System.out.print("Sum Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(" " + sum[i]);
        }
        System.out.println();
        sc.close();
    }
}
```

Op:

enter no.of elements: 7

enter elements: 5 8 7 3 4 5 6

Sum Array: 11 13 11 6 11 13 11

ALTERNATIVE:

```
import java.util.Scanner;

public class alternativeofsumreverse {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
```

```

        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        for (int i = 0; i < n; i++) {
            System.out.println(arr[i] + arr[n - i - 1] + " ");
        }
        sc.close();
    }
}

```

Op:

enter no.of elements7

enter elements: 5 8 7 3 4 5 6

11 13 11 6 11 13 11

COUNT OF ELEMENTS THAT ARE LESS THAN EACH ELEMENT IN ARRAY:

```

import java.util.Scanner;
public class smallerthaneachelement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++)
        {
            a[i] = sc.nextInt();
        }
        for (int j = 0; j < n; j++) {
            int m = 0;
            for (int k = 0; k < n; k++) {
                if (a[j] > a[k]) {
                    m++;
                }
            }
            System.out.print(m + " ");
        }
        sc.close();
    }
}

```

Op:

8

5 8 3 5 2 2 1 6

4 7 3 4 1 1 0 6

PRINT UNIQUE ELEMENTS (I.E REMOVE DUPLICATES):

```
import java.util.Scanner;
public class uniqueelementsofarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int[] res = new int[n];
        int k = 0;
        for (int i = 0; i < n; i++) {
            int flag = 0;
            for (int j = 0; j < k; j++) {
                if (res[j] == arr[i]) {
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
                res[k] = arr[i];
                k++;
            }
        }
        for (int i = 0; i < k; i++) {
            System.out.print(res[i] + " ");
        }
        sc.close();
    }
}
```

Op:

enter no.of elements: 15

enter elements: 6 7 7 5 6 3 4 2 3 5 2 1 1 1 2

6 7 5 3 4 2 1

Alternative:

```
import java.util.Scanner;
public class alteruniquearray {
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("enter no.of elements: ");
    int n = sc.nextInt();
    int[] arr = new int[n];
    int[] hash = new int[101];
    int[] res = new int[n];
    int k = 0;
    System.out.print("enter elements: ");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }
    for (int i = 0; i < n; i++) {
        if (hash[arr[i]] == 0) {
            res[k++] = arr[i];
            hash[arr[i]] = 1;
        }
    }
    for (int i = 0; i < k; i++) {
        System.out.print(res[i] + " ");
    }
    sc.close();
}

```

Op:

enter no.of elements: 15

enter elements: 6 7 7 5 6 3 4 2 3 5 2 1 1 1 2

6 7 5 3 4 2 1

=== Code Execution Successful ===

FIRST ELEMENT THAT IS GREATER THAN X:

```

import java.util.Scanner;
public class greaterthanxarray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.print("enter element you want to find the just greater value: ");
        int x = sc.nextInt();
    }
}

```

```

        for (int i = 0; i < n; i++) {
            if (arr[i] > x) {
                System.out.print(arr[i]);
                break;
            }
        }
        sc.close();
    }
}

```

Op:

enter no.of elements: 6

enter elements: 3 5 8 2 4 1

enter element you want to find the just greater value: 4 5

Remove duplicates and print the array:

```

import java.util.Scanner;
import java.util.Arrays;;

public class removeduplicates {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int[] res = new int[n];
        int k = 0;
        for (int i = 0; i < n; i++) {
            int f = 0;
            for (int j = 0; j < k; j++) {
                if (arr[i] == res[j]) {
                    f = 1;
                    break;
                }
            }
            if (f == 0)
                res[k++] = arr[i];
        }
        System.out.print(Arrays.toString(res));
        sc.close();
    }
}

```

20-09-2025 ,
Saturday

♦ 2D ARRAY:

Declaration and initialization:

```
int[][] arr=new int[n][m];
for(int i=0;i<n;i++){           //rows
    for(int j=0;j<m;j++){       //columns first j runs
        arr[i][j]=sc.nextInt();
    }
}
```

Print Elements of 2D array:

```
import java.util.Scanner;
public class twodarrayinput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements in row and column: ");
        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

enter no.of elements in row and column: 4 6

enter elements:

4 5 6 7 8 9

3 4 5 2 1 1

5 5 4 4 3 3

6 7 8 5 3 2

4 5 6 7 8 9

3 4 5 2 1 1

5 5 4 4 3 3

6 7 8 5 3 2

Count of odd elements in row:

```
import java.util.Scanner;

public class countoddinrow2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements in row and column: ");
        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (arr[i][j] % 2 != 0) {
                    count++;
                }
            }
            System.out.println("count in " + i + " is " + count);
        }
        sc.close();
    }
}
```


Even count of elements in 2D:

```
import java.util.Scanner;

public class evencount2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements in row and column: ");
        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (arr[i][j] % 2 == 0) {
                    count++;
                }
            }
        }
        System.out.print(count);
        sc.close();
    }
}
```

SEARCH ELEMENT:

```
import java.util.Scanner;

public class searchelementin2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements in row and column: ");
        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }

        System.out.print("enter element to found: ");
        int found = sc.nextInt();
    }
}
```

```

int f = 0;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (arr[i][j] == found) {
            f = 1;
            System.out.print("yes");
            break;
        }
    }
}

if (f == 0)
    System.out.print(" not found");
sc.close();
}
}

```

SUM OF ELEMENTS IN COLUMN:

```

import java.util.Scanner;

public class umofelementsincoumn2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements in row and column: ");
        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        int sum = 0;
        for (int i = 0; i < m; i++) {

            for (int j = 0; j < n; j++) {
                sum += arr[j][i];
            }
            System.out.println(sum);
        }
        sc.close();
    }
}

-----

import java.util.Scanner;
public class transposematrix {
    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);
System.out.print("enter no.of elements in row and column: ");
int n = sc.nextInt();
int m = sc.nextInt();
int[][] arr = new int[n][m];
System.out.print("enter elements: ");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        arr[i][j] = sc.nextInt();
    }
}
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        System.out.print(arr[j][i] + " ");
    }
    System.out.println();
}
sc.close();
}}

```

enter no.of elements in row and column: 3 3

enter elements: 4 6 8

2 3 4

1 4 2

4 2 1

6 3 4

8 4 2

Print the element in given no of positions:

```

import java.util.Scanner;
public class outputpositionin2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements in row and column: ");
        int n = sc.nextInt();
        int m = sc.nextInt();
        int[][] arr = new int[n][m];
        System.out.print("enter elements in row and column: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        System.out.print("enter outputs: ");
        int t = sc.nextInt(); // number of test cases
        while (t != 0) {
            int pos = sc.nextInt();

```

```

        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (count == pos) {
                    System.out.print(arr[i][j] + " ");
                    break;
                }
                count++;
            }
        }
        t--;
    }
    sc.close();
}

// int t=sc.nextInt();
// while(t!=0){
//     int pos=sc.nextInt();
//     sop(arr[pos/m][pos%m]);
//     t--;}

```

op:

enter no.of elements in row and column: 3 4

enter elements in row and column: 5 6 8 3

6 2 7 8

5 6 1 2

enter outputs: 3

6

10

11

7 5

1 2

11-10-2025,
Saturday

2D ARRAY PROBLEMS

Assume raghu playing game has to select 3 boxes i.e 1.upper triangular matrix

2.diagonal matrix

3.lower triangular matrix

To win in the game he has to select the box that has maximum sum of elements

Now the sum of the elements of the box selected should greater than others
Help raghu find the maximum sum of elements to select 3 boxes from these upper triangular matrix ,lower triangular matrix,diagonal matrix

```
import java.util.*;
public class sumof3boxes2darray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no of elements: ");
        int n = sc.nextInt();
        int[][] arr = new int[n][n];
        System.out.print("enter no elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
            System.out.println();
        }
        int sum1 = 0, sum2 = 0, sum3 = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j)
                    sum1 += arr[i][j];
                else if (i < j)
                    sum2 += arr[i][j];
                else
                    sum3 += arr[i][j];
            }
        }
        if (sum1 > sum2 && sum1 > sum3)
            System.out.println("Diagonal Matrix:" + sum1);
        if (sum2 > sum3 && sum2 > sum1)
```

```

        System.out.println("upper triangular matrix:" + sum2);
    if (sum3 > sum2 && sum3 > sum1)
        System.out.println("lower triangular matrix : " + sum3);
    sc.close();
    }
}

```

Ip:

enter no of elements: 4

enter no elements:

3 1 1 3

4 9 1 2

2 1 8 3

4 2 6 9

Op:

Diagonal Matrix:29

enter no of elements: 4

enter no elements:

16 15 14 13

12 11 10 9

8 7 6 5

4 3 2 1

upper triangular matrix:66

◆Array rotation

1.Print the 1D array elements after no of rotations:

Ip:

6

1 2 3 4 5 6

2

```

import java.util.Scanner;
public class arrayrotationprint {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter " + n + " elements: ");
    }
}

```

```

    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }
    System.out.print("Enter number of rotations: ");
    int rotationCount = sc.nextInt();
    for (int r = 0; r < rotationCount; r++) {
        int last = arr[n - 1];
        for (int i = n - 1; i > 0; i--) {
            arr[i] = arr[i - 1];
        }
        arr[0] = last;
    }
    System.out.print("Array after " + rotationCount + " rotations: [");
    for (int i = 0; i < n; i++) {
        System.out.print(arr[i]);
        if (i < n - 1)
            System.out.print(", ");
    }
    System.out.println("]");
    sc.close();
}
}

```

2.Alternative: timecomplexity $O(n/2)$

```

import java.util.Scanner;
public class ArrayRotationPrintalter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter " + n + " elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.print("Enter number of rotations: ");
        int rotationCount = sc.nextInt();
        for (int r = 0; r < rotationCount; r++)
        {
            int last = arr[n - 1];
            for (int i = n - 1; i > 0; i--) {
                arr[i] = arr[i - 1];
            }
            arr[0] = last;
        }
        System.out.print("Array after " + rotationCount + " rotations: [");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i]);
            if (i < n - 1)
                System.out.print(", ");
        }
    }
}

```

```

    }
    System.out.println("]");
    sc.close();    }    }

```

3.Alternative:

```

import java.util.Scanner;
public class arrayrotationprint {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter " + n + " elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.print("Enter number of rotations: ");
        int r = sc.nextInt();
        r = r % n;
        System.out.print("[");
        for (int i = n - r; i < n; i++) {
            System.out.print(arr[i]);
            System.out.print(", ");
        }
        for (int i = 0; i < n - r; i++) {
            System.out.print(arr[i]);
            if (i < n - r - 1)
                System.out.print(", ");
        }

        System.out.println("]");
        sc.close();    }    }

```

4.Alternative:

```

import java.util.Arrays;
import java.util.Scanner;
public class reversearrayrotation1d {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            int n = sc.nextInt();
            int[] arr = new int[n];
            int[] rev = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            int t = sc.nextInt();
            int p = n - 1;
            for (int i = 0; i < n; i++) {
                rev[i] = arr[p];
                p--;
            }
        }
    }
}

```



```

    }
    for (int i = 0; i < t - 1 && i + 1 < n; i++) {
        int temp = rev[i];
        rev[i] = rev[i + 1];
        rev[i + 1] = temp;
    }
    Arrays.sort(rev, t, n);
    for (int i = 0; i < n; i++) {
        System.out.print(rev[i] + " ");
    }
}
}
}

```

Alternative:

Else logic:

```
Int rotationcount=sc.nextInt();
```

```
Int[] res=new int[n];
```

```
Int x=0;
```

```
for(int i=n;i<rotationcount;i++) {
    res[x++]=arr[i];
}
```

```
for(int i=0;i<(n-rotationcount);i++){
    res[x++]=arr[i];
}
```

(or)

timecomplexity:

$O(n*n)$

6.Alternative:

```
while(rotationcount!=0)
```

```
{
```

```
Int t=arr[n-1];
```

```
for(int i=n-1;i>=0;i--)
```

```
{
```

```
    if(i!=0)
```

```
        arr[i]=arr[i-1];
```

```
    Else
```

```
        arr[i]=t;
```

```
    rotationcount--;
```

```
}
```

(or)

timecomplexity:

$O(\text{rotationcount}*n)$

spacecomplexity:

$O(1)$

7.Alternative:

```
Int n=arr.length;
```

```
steps=steps%n;
```

```
for(int i=0;i<n;i++){
```

```
    if(steps>0)
```

```

        res[(i+steps)%n]=arr[i];
    Else
        res[(i+steps+n)%n]=arr[i];
}

```

```
System.out.print(res+" ");
```

Op:

Enter number of elements: 6

Enter 6 elements: 1 2 3 4 5 6

Enter number of rotations: 2

[5, 6, 1, 2, 3, 4]

Anticlock wise array:

```

import java.util.Scanner;
public class degreeerotate2d90 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements: ");
        int n = sc.nextInt();
        int[][] arr = new int[n][n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        int[][] r = new int[n][n];
        int a = 0;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j < n; j++) {
                r[a][j] = arr[i][j];
            }
            a++;
        }
        System.out.println("-----");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.err.print(r[j][i] + " ");
            }
            System.err.println();
        }
        sc.close();
    }
}

```

Op:

enter no.of elements: 4

enter elements:

1 2 3 4

5 6 7 8

9 3 4 6

3 2 1 1

3 9 5 1

2 3 6 2

1 4 7 3

1 6 8 4

180 degrees rotate:

```
import java.util.Scanner;
public class degree180rotate2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] arr = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        System.err.println("");
        int[][] r = new int[n][n];
        int p = 0;
        for (int i = n - 1; i >= 0; i--) {
            int a = n - 1;
            for (int j = 0; j < n; j++) {
                r[p][j] = arr[i][a];
                a--;
            }
            p++;
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.err.print(r[i][j] + " ");
            }
            System.err.println();
        }
        sc.close();    }
}
```

Op:

3

1 2 3

4 5 6

7 8 9

9 8 7

6 5 4

3 2 1

Reverse of rotate of 180 degrees:

```
import java.util.Scanner;
public class rotatereverse2d {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements: ");
        int n = sc.nextInt();
        int[][] arr = new int[n][n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        int[][] r = new int[n][n];
        int a = 0;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j < n; j++) {
                r[a][j] = arr[j][i];
            }
            a++;
        }
        System.out.println("-----");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.err.print(r[i][j] + " ");
            }
            System.err.println();
        }
        sc.close();
    }
}
```

270 degrees rotate:

```
import java.util.Scanner;
public class degreeerotate2d90 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("enter no.of elements: ");
        int n = sc.nextInt();
        int[][] arr = new int[n][n];
        System.out.print("enter elements: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = sc.nextInt();
            }
        }
        int[][] r = new int[n][n];
        int a = 0;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j < n; j++) {
                r[a][j] = arr[i][j];
            }
            a++;
        }
        System.out.println("-----");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                {
                    System.err.print(r[j][i] + " ");
                }
            }
            System.err.println();
        }
        sc.close();
    }
}
```

17-10-25,
Friday;

◆ STRINGS

Declaration of strings:

```
String str="hello";
String str1="hello";
```

// points to same location because they are same;
// in arrays i.e integer it is problem here it is not problem

//strings are immutable

✓ STRING FUNCTIONS:

1.str.length():find length of string

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = "hello";

        System.out.println(str.length());           //op:5
        sc.close();
    }
}
```

2.str.charAt():find character at location

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = "hello";
        System.out.println(str.charAt(1));
        sc.close();
    }
}
```

Op:e

3.str.indexOf():find the index of value

```
import java.util.Scanner;
```

```
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = "hello";
        System.out.println(str.indexOf("l"));
        System.out.println(str.lastIndexOf("l"));
        System.out.println(str.indexOf("ell"));
        sc.close();
    }
}
```

str.replace(char,with):replace one char with another

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = "hello";
        System.out.println((str.replace("l", "p")));
        System.out.println((str.toUpperCase()));
        sc.close();
    }
}
```

Op:

heppo

str.toUpperCase:change to upper case

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = "hello";
        System.out.println((str.toUpperCase()));
        sc.close();
    }
}
```

Op:

HELLO

Str.equals:checks whether both are equal or not

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = "hello";
```

```
String s2 = "HELLO";
System.out.println(s1.equals(s2));
sc.close();
}
}
```

Op:

False

str.equalsIgnoreCase: ignores the case

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = "hello";
        String s2 = "HELLO";
        System.out.println(s1.equals(s2));
        System.out.println(s1.equalsIgnoreCase(s2));
        sc.close();
    }
}
```

Op:

true

str.trim(): remove the spaces in the string

```
import java.util.Scanner;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s3 = "    hi    ";
        System.out.println(s3);
        System.out.println(s3.trim());
        sc.close();
    }
}
```

Op:

hi

hi

Str.split: it will split into individual

```
import java.util.Scanner;
```



```
import java.util.Arrays;
public class stringbasic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s4 = "hey all how are you";
        System.out.println(Arrays.toString(s4.split(" ")));
        sc.close();
    }
}
```

Op:

[hey, all, how, are, you]

```
public class stringbasic {
    public static void main(String[] args) {

        String s4 = "hey all how are you";
        System.out.println(Arrays.toString(s4.split(" ")));
        System.out.println(Arrays.toString(s4.split("a")));
    }
}
```

Op:

[hey , ll how , re you]

StringBuilder:

```
public class stringbuilder {
    public static void main(String[] args) {
        StringBuilder str = new StringBuilder("Hello");
        System.out.print(str);
    }
}
```

Op:

Hello

str.append(): change and return the string

```
public class stringbuilder {
    public static void main(String[] args) {
        StringBuilder str = new StringBuilder("Hello");
        System.out.print(str);
        System.out.println(str.append("123"));    }
}
```

Op:

HelloHello123

Str.insert:insert abc at 5th position

```
public class stringbuilder {
    public static void main(String[] args) {
        StringBuilder str = new StringBuilder("Hello");
        System.out.println(str);
        System.out.println(str.append("123"));
        System.out.println(str);
        str.insert(5, "abc");
        System.out.println(str);
    }
}
```

Op:

Hello

Hello123

Hello123

Helloabc123

1.To count no. of vowels in the given string:

```
import java.util.Scanner;
public class printvowelsinstring {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        int count = 0;

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
                ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
                count++;
            }
        }
        System.out.println("Number of vowels: " + count);
        sc.close();
    }
}
```

Op:

Enter a string: hippopotamus

Number of vowels: 5

2.print vowels as a string:

Ip: hippopotamus—> i o o a u

```
import java.util.Scanner;
public class stringbuildervowel {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        StringBuilder str = new StringBuilder();

        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
                ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
                str.append(ch+" ");
            }
        }
        System.out.println(str.toString());
        sc.close();
    }
}
```

Op:

Enter a string: hippopotamus

i o o a u

3.print count of uppercase and lowercase char in given string:

```
import java.util.Scanner;
public class countlowerupperinstring {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        int upperc = 0, lowerc = 0;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch)) {
                upperc++;
            }
        }
    }
}
```

```

        } else if (Character.isLowerCase(ch)) {
            lowerc++;
        }
    }
    System.out.println(upperc);
    System.out.println(lowerc);
    sc.close();
}
}

```

Op:

Enter a string: helloWORLD

5

5

Enter a string: KAMUNlakshaya

6 7

4.give a string with upper,lowercase,digits,splchar.
Print count of alphabets,digits and spl char

```

import java.util.Scanner;
public class countlowerupperdigitsplchar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        int uc = 0;
        int lc = 0;
        int dc = 0;
        int splc = 0;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch)) {
                uc++;
            } else if (Character.isLowerCase(ch)) {
                lc++;
            } else if (Character.isDigit(ch)) {
                dc++;
            } else {
                splc++;
            }
        }
        System.out.println("Uppercase letters: " + uc);
    }
}

```

```

        System.out.println("Lowercase letters: " + lc);
        System.out.println("Digits: " + dc);
        System.out.println("Special characters: " + splc);
        sc.close();
    }
}

```

Op:

Enter a string: spITGAr1903@!!

Uppercase letters: 3

Lowercase letters: 4

Digits: 4

Special characters: 3

5. "axcbye" —> [1,26,3,2,25,5]

print positions of alphabets

```

import java.util.Scanner;
public class alphabetposition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        StringBuilder positions = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isLetter(ch)) {
                int pos = Character.toLowerCase(ch) - 'a' + 1;
                positions.append(pos).append(" ");
            }
        }
        System.out.println("Alphabet positions: " + positions.toString().trim());
        sc.close();
    }
}

```

Op:

Enter a string: abdhbx

Alphabet positions: 1 2 4 8 2 24

6. password checker(atleast 1 upper,1 lower ,1digit,1 splchar,length>=8)

7.lower,upper,digit count in string

```
import java.util.Scanner;
public class countonlylowerupperdigit {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        int uc = 0;
        int lc = 0;
        int dc = 0;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isUpperCase(ch)) {
                uc++;
            } else if (Character.isLowerCase(ch)) {
                lc++;
            } else if (Character.isDigit(ch)) {
                dc++;
            }
        }
        System.out.println("Uppercase letters: " + uc);
        System.out.println("Lowercase letters: " + lc);
        System.out.println("Digits: " + dc);
        sc.close();
    }
}
```

op:Enter a string: fgsRTsdf3gg2fsd4r53

Uppercase letters: 2

Lowercase letters: 12

Digits: 5

(or)

int u=0,l=0,d=0;

for(int i=0;i<str.length();i++){

 char ch=str.charAt(i); A-65

 if(ch>=65&&ch<=90) Z-90

 u++; a-97

 else if(ch>=97&&ch<=122) z=122

```

        L++;
else
    d++;}

```

0-48
9-57

8.place all lower,upper,digits at one place append in one string itself

```

import java.util.Scanner;
public class printupperlowerdigitsappendatsingleplace {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = sc.nextLine();
        StringBuilder upperCase = new StringBuilder();
        StringBuilder lowerCase = new StringBuilder();
        StringBuilder digits = new StringBuilder();
        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);
            if (Character.isUpperCase(ch)) {
                upperCase.append(ch);
            } else if (Character.isLowerCase(ch)) {
                lowerCase.append(ch);
            } else if (Character.isDigit(ch)) {
                digits.append(ch);
            }
        }
        System.out.println(upperCase.toString() + lowerCase.toString() +
digits.toString());
        sc.close();
    }
}

```

Op:

Enter a string: akndGY76bdkEG

GYEGakndbdk76

9.Alphabet position in array format:

```

import java.util.Arrays;
import java.util.Scanner;
public class aplhabetpoistioninarrayform {

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String s = sc.nextLine();
    StringBuilder positions = new StringBuilder();
    for (int i = 0; i < s.length(); i++) {
        char ch = s.charAt(i);
        if (Character.isLetter(ch)) {
            int pos = Character.toLowerCase(ch) - 'a' + 1;
            positions.append(pos).append(" ");
        }
    }
    String[] posArray = positions.toString().trim().split(" ");
    System.out.println("Alphabet positions ");
    System.out.println(Arrays.toString(posArray));
    sc.close();
}

```

Op:

Enter a string: azbxcy

Alphabet positions: [1 ,26 ,2 ,24 ,3 ,25]
(or)

String str=sc.nextLine();

Int[] res=new int[str.length()];

for(int i=0;i<str.length();i++){
 res[i]=str.charAt(i)-'a'+1;

}

sop(Arrays.toString(res));

10.Print alphabets for given array elements

```

import java.util.Scanner;
public class returnalphabetsofarrayelementsofnumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the Array Count: ");
        int n = sc.nextInt();
        int arr[] = new int[n];
        System.out.print("Enter Array Elements: ");
    }
}

```



```

    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }
    String str = "";
    for (int i = 0; i < n; i++) {
        str += (char) (arr[i] + 96);
    }
    System.out.print(str);
    sc.close();
}
}

```

Op:

Enter the Array Count:

6

Enter Array Elements: 7 12 4 7 3 9

gldgci

11.Coded-encoded: school-hxsllo hgfwvmg-student

```

public class SimpleEncode {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the String: ");
        String str = sc.nextLine();
        String res = "";
        for (int i = 0; i < str.length(); i++) {
            res += (char) (219 - str.charAt(i));
        }
        System.out.print(res);
        sc.close();
    }
}

```

Op:

Enter the String:

hgfwvmg

student

12.coded-encoded

Ip:khoo

3

Op:hello

Move 3 steps back of k and print that alphabet and 3 steps from h and print that alphabet and so on

```
import java.util.Scanner;
public class encodedstring {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the String: ");
        String str = sc.nextLine();
        System.out.print("Enter the number to be subtracted: ");
        int n = sc.nextInt();
        String res = "";
        for (int i = 0; i < str.length(); i++)
        {
            if (str.charAt(i) - n < 'a')
                res += (char) (str.charAt(i) - n + 26);
            else
                res += (char) (str.charAt(i) - n);
        }
        System.out.print(res);
        sc.close();
    }
}
```

Op:

Enter the String:

bvec

Enter the number to be subtracted: 4

xray

(or)

```
for (int i = 0; i < str.length(); i++) {
    if (str.charAt(i) - n >= 97) {
```

```

        res += (char) (str.charAt(i) - n);
    } else {
        res += (char) (str.charAt(i) - n + 26);
    }
}

```

Ip:aaabbbaaaadddcbbb

Op:a3b2a4d3c1b3

String compression and print:

```

import java.util.Scanner;
public class stringcompressionprint {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the String: ");
        String str = sc.nextLine();
        String res = "";
        int count = 1;
        for (int i = 0; i < str.length(); i++) {
            if (i < str.length() - 1 && str.charAt(i) == str.charAt(i + 1)) {
                count++;
            } else {
                res += str.charAt(i);
                if (count >= 1) {
                    res += count;
                    count = 1;
                }
            }
        }
        System.out.print(res);
        sc.close();
    }
}

```

Op:

Enter the String:

aaaabjjjjssssssssbb

a4b1j4s8b2

Enter the String:

nnnnnnnnnddhhh
n8d2h3

(or)

```
for(int i=0;i<str.length;i++){
    if(str.charAt(i)==str.charAt(i+1))
        c++;
    else{
        res+=str.charAt(i)+c;
        c=1;
    }
}
res=res+str.charAt(str.length()-1)+c;
```

(or)

```
for(int i=0;i<str.length;i++){
    Char ch=str.charAt(i);
    Char ch1=str.charAt(i+1);
    if(ch==ch1)        c++;
    else{
        res+=ch+c;
        c=1;
    }
}
res=res+str.charAt(str.length()-1)+c;
```

Password checker:

It should contain 8 characters ; at least one lowercase ,1 uppercase,1special character,1 digit

Now print the count of what all are missing

i/p : Qwertyui

o/p : 2

i/p : 123

o/p : 5

i/p : 1234567

o/p : 3

i/p : Abc\$aw

o/p : 2

i/p : Ab@2

o/p : 4

```
import java.util.*;
public class passwordcheckercount {
    public static void main(String[] args) {
        Scanner ss = new Scanner(System.in);
        String p = ss.nextLine();
        int len = 8;
        int lc = 0, uc = 0, d = 0, sc = 0;
        for (int i = 0; i < p.length(); i++) {
            char ch = p.charAt(i);
            if (Character.isLowerCase(ch))
                lc++;
            else if (Character.isUpperCase(ch))
                uc++;
            else if (Character.isDigit(ch))
                d++;
            else
                sc++;
        }
        if (lc >= 1 && uc >= 1 && d >= 1 && sc >= 1 && p.length() >= len) {
            System.out.println("Valid Password");
        } else {
            int ps = 0;
            if (lc == 0)
                ps++;
            if (uc == 0)
                ps++;
            if (d == 0)
                ps++;
            if (sc == 0)
                ps++;
            if (p.length() < len) {
                int m = len - p.length();
                if (ps < m)
                    ps = m;
            }
        }
    }
}
```

```

    }
    System.out.println(ps);
}
ss.close();    }
}

```

sdabcdeghijruvwxyzpqrstabc

o/p : 6

i/p : baxzqpwvu

o/p : 1

i/p : hijklmnop

o/p : 9

import java.util.*;

Class Main {

public static void main(String[] args) {

System.out.println("Enter the string: ");

Scanner s = new Scanner(System.in);

String str = s.nextLine();

int MaxCount = 1;

int count=1;

for(int i=0;i<str.length()-1;i++)

{

char c = str.charAt(i);

char c1 = str.charAt(i+1);

if(c1-c == 1)

count++;

else

{

if(count>MaxCount)

{

MaxCount=count;

count=1;

}

else

```

        count=1;
    }
}
if(count>MaxCount)
    MaxCount=count;
System.out.print(MaxCount);
}
}

```

question:

i/p : ()()()())(

o/p : 5

i/p :)))))))()(((

o/p : 7

i/p :))((

o/p : 2

i/p : (()()())()

o/p : 5

i/p : ((((((

o/p : 0

(output is the equal point where number of open braces before that point and number of closed after that point including the point)

```
import java.util.*;
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Enter the string: ");
```

```
        Scanner s = new Scanner(System.in);
```

```
        String str = s.nextLine();
```

```
        int count=0;
```

```
        for(int i=0;i<str.length();i++)
```

```
        {
```

```

        if(str.charAt(i)=='')
            count++;
    }
    System.out.println(count);
}
}

```

15-11-2025;
Saturday;

SORTING

● BUBBLE SORT :

Used for sorted arrays

- Bubble Sort is a simple comparison-based sorting algorithm in which adjacent elements are compared and swapped if they are in the wrong order.
- It repeatedly "bubbles" the largest (or smallest) element to the end of the array.

✓ Conditions / Key Points About Bubble Sort

① Repeatedly compare adjacent elements

For index i and $i+1$, compare:

```

if (arr[i] > arr[i+1])
    swap(arr[i], arr[i+1]);

```

② After each pass, one largest element reaches the correct position

- After 1st pass → largest element is at last index

- After 2nd pass → 2nd largest is at second last index AND so on.....

3 Number of passes

For an array of size n , Bubble Sort needs $n-1$ passes.

4 Optimized version stops early

If in a full pass no swaps happen, the array is already sorted → stop early
This saves time for nearly sorted arrays.

5 Time Complexity

Case	Time Complexity	Reason
Best	$O(n)$	Already sorted, 1 pass
Avg	$O(n^2)$	Random order
Worst	$O(n^2)$	Reverse sorted
Space	$O(1)$	In-place algorithm

6 Stable Sort

Bubble sort is stable → does not change the order of equal elements.

7 Not suitable for big inputs

Because of $O(n^2)$ time, used only for:

Teaching sorting concepts, Small inputs, Almost sorted data

 **When to Use Bubble Sort**

Use when:

- ✓ Teaching beginners
- ✓ Very small datasets
- ✓ Checking if array is almost sorted

Avoid when:

- ✖ Large datasets
- ✖ Performance-critical tasks

CODE:

```
import java.util.Arrays;
import java.util.Scanner;

public class bubblesort {

    public static void main(String[] args) {
        System.out.println("Enter the array length: ");
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter Array Elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        sc.close();
        for (int i = 0; i < n - 1; i++) {
            int flag = 0;
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    flag = 1;
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
            if (flag == 0) {
                break;
            }
        }
        System.out.print(Arrays.toString(arr));
    }
}
```

(or)

```
public class phone {
    static void bubbleSort(int brr[]) {
```

```

int n=brr.length;
for(int spot=0;spot<n;spot++){
    for( int adj=spot+1;spot<n;adj++){
        if(brr[spot]>brr[adj]){
            Int temp=brr[spot];
            brr[spot]=brr[adj];
            brr[adj]=temp;
        }
    }
}
}
public static void main(String[] args){
    //
    int arr[]={-1,3,2,4,1,56,0}
    int b=12;
    bubbleSort();    }

```

Example:

{-1,3,2,4,1,56,0}

- Operations: took are 21
- But actually by using $O(n)$ it is 49 operations:
- Sample: after sort operations using loop :{ -1,0,1,2,3,4,56}
- Uses pass by reference-modifies the original value pass memory
- Pass by value- a new memory is created when a new value is given

Selection sort:

Selection Sort is a simple comparison-based sorting algorithm.

It sorts an array by **repeatedly selecting the smallest element** from the unsorted part and placing it at the beginning.

How It Works (Step-by-step)

1. Start from the first element.
2. Find the **minimum** element in the entire array.

3. Swap it with the first element.
4. Move to the next position and repeat.
5. Continue until the whole array becomes sorted.

Example

Array: [5, 2, 8, 1, 3]

- Pass 1 → smallest is 1 → swap with first
→ [1, 2, 8, 5, 3]
- Pass 2 → smallest in remaining (2,8,5,3) is 2
→ [1, 2, 8, 5, 3]
- Pass 3 → smallest in (8,5,3) is 3
→ [1, 2, 3, 5, 8]
- Pass 4 → smallest in (5,8) is 5
→ already in place

Time Complexity

Case	Complexity
Best Case	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$

👉 It always performs the same number of comparisons, even if the array is already sorted.

Space Complexity

- $O(1)$ (in-place sorting)

- Very memory efficient because it does not use extra arrays.

Key Features

- Simple to implement
- Not very efficient for large datasets. Good for small lists
- Performs **minimum number of swaps** (better than bubble sort in swapping operations)

Selection Sort Algorithm (Pseudo-code)

```
for i from 0 to n-2:
    minIndex = i
    for j from i+1 to n-1:
        if array[j] < array[minIndex]:
            minIndex = j
    swap array[i] and array[minIndex]
```

Code:

```
import java.util.Arrays;
import java.util.Scanner;

public class selectionsort {
    public static void main(String[] args) {
        System.out.println("Enter the array length: ");
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter Array Elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        sc.close();
        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }
            swap(arr, i, minIndex);
        }
        System.out.println("Sorted Array: ");
        Arrays.sort(arr);
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }

    static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

```

    }
}
    if (minIndex != i) {
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
System.out.print(Arrays.toString(arr));
}
}

```

(or)

```

public class example {
static void selectionSort(int brr[]) {
    int n=brr.length;
    for(int i=0;i<n-1;i++){
        for(int j=n+1;j<n;j++){
            if(brr[j]<minindex){
                minindex=j;
            }
        }
        if(minindex!=i){
            Int temp=brr[i];
            brr[i]=brr[minindex];
            brr[minindex]=temp;
        }
    }
}
public static void main(String[] args){
int arr[]={-1,3,2,4,1,56,0}
int b=12;
selectionSort();
}

```

Insertion sort:

Insertion Sort builds the final sorted array **one element at a time**.

It takes each element and **inserts it into the correct position** in the already sorted part of the array.

It works similar to how you sort cards in your hand.

How It Works (Step-by-step)

1. Start from the second element (index 1) — the first element is considered **sorted**.
2. Compare the current element with the elements before it.
3. Shift all larger elements one position to the right.
4. Insert the current element into the correct position.
5. Repeat for all elements.

ex: Array: **[5, 2, 4, 6, 1]**

- Take 2 → insert before 5
→ [2, 5, 4, 6, 1]
- Take 4 → insert between 2 and 5
→ [2, 4, 5, 6, 1]
- Take 6 → correct position
→ [2, 4, 5, 6, 1]
- Take 1 → insert at the beginning
→ [1, 2, 4, 5, 6]

Best Case (already sorted)	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity

$O(1)$ (in-place) No extra memory needed.

Key Features

- Simple and intuitive
- Efficient for **small** or **nearly sorted** arrays
- Stable sorting algorithm (keeps equal elements in original order)

Insertion Sort Algorithm (Pseudo-code)

```

for i from 1 to n-1:
    lost= array[i]
    j = i - 1
    while j >= 0 and array[j] >lost:
        array[j + 1] = array[j]
        j = j - 1
    array[j + 1] = lost

```

Code:

```

import java.util.Arrays;
import java.util.Scanner;

public class insertionsort {
    public static void main(String[] args) {
        System.out.println("Enter the array length: ");
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter Array Elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }
}

```



```

sc.close();
for (int i = 1; i < n; i++) {
    int lost= arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] >lost) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] =lost;
}
System.out.print(Arrays.toString(arr));
}
}

```

Logic:

```

i from for 0 to n
    lost= arr[j]
    i = j + 1
    while j > 0 and arr[j] >lost:
        arr[j + 1] = arr[j]
        j = j + 1
    arr[j + 1] = lost

```

Logic algorithm:

- Run the subarray 0 to n-1
- Write the logic for ins in sort sub array
- Lost var --- i will lost it so i need t
- Ti var — i dont want to mess up with i
- J var – inserting element
- Run the comparisons with lost and ti and stop it if u go beyond 0
- Do the shift from ti to ti+1
- Then decrement ti
- Finally insert the lost element in ti

(or)

```

public class InsertionSort {
    public static void insertionSort(int[] arr) {
        Int n=arr.length;
        for (int i = 0; i <n-1; i++) {

```

```

    Int ti=i;
    Int lost=arr[j];
    Int j=i+1;
    for(; ti >=0 && lost<arr[ti];){
        arr[ti+1]=arr[ti];
        ti- - ;
    }
    arr[ti+1]=lost;
}
}
public static void main(String[] args) {
    int[] arr = {2,10,23,34,40,13,76};
    insertionSort(arr);
    System.out.print("Sorted array: ");
    for (int n : arr)
        System.out.print(n + " ");
}
}

```

Find second largest element in array:

```

public class phone    {
static void bubbleSort(int brr[])  {
    int n=brr.length;
    for(int spot=n-1;spot>=n-2;spot- -){
        for( int adj=spot-1;spot>=0;adj- -){
            if(brr[spot < brr[adj]){
                int temp=brr[spot];
                brr[spot]=brr[adj];
                brr[adj]=temp;
            }
        }
    }
}
}
}

```

```

public static void main(String[] args){
    //
    int arr[]={-1,3,2,4,1,56,0}
    int b=12;
    bubbleSort(); }

```

Asymptotic notations:

Big -O	worst case	at worst ,how bad can it be
Omega	best case	at best how fast can it be
Theta	exact bound	it always performs

Why we prefer Big-O in real life

We care about -> Worst case scenario—

Even in worst case,performance won't be worse than this

Practical safe

Best case example:

suppose[5,2,9,1,7]

9 in 1st position ->only 1 comparison

So omega(1)-> constant time

If u want to deal with Worst case -> think about all elements

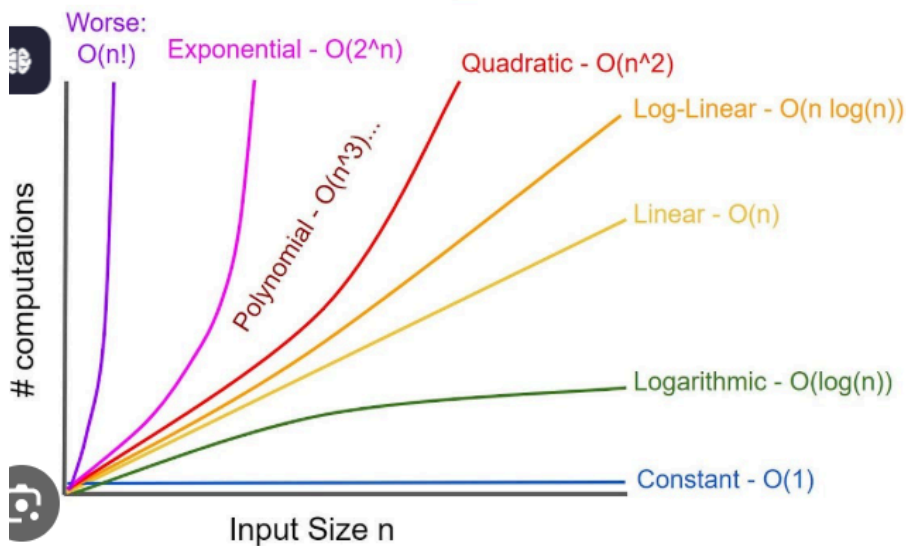
All time complexities:

Time Complexity	Notation	Typical Operations / Examples
Constant Time	$O(1)$	Accessing array element, simple math ops, hash lookup
Logarithmic	$O(\log n)$	Binary search, tree operations (balanced BST)

Linear	$O(n)$	Linear search, scanning an array
Log-Linear	$O(n \log n)$	Merge sort, Quick sort (avg), Heap sort
Quadratic	$O(n^2)$	Nested loops, Bubble sort, Insertion sort (worst), many delete/insert operations in arrays
Polynomial	$O(n^3)$, $O(n^4)$...	3 nested loops, matrix multiplication (n^3), complex DP
Exponential	$O(2^n)$	Recursive Fibonacci, subsets, combinations generation
Factorial (Worst)	$O(n!)$	Permutations generation, Traveling Salesman brute force

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

Best -----avg-----worst
Worst (not able to handle more values)



This best ,avg,worst for coders i.e whether good coder or not ,it is not about the no of operations it took complexities

5-1-2026;

Monday;

TIME COMPLEXITY

Basic rules for time complexity:

1. Remove the constant values
2. Retain the higher order values

Example:

```
public class sumarray {
    public static void main(String[] args) {
        int[] arr={3,5,3,6,9} -----1
        int sum = 0; -----1
        int n=arr.length -----1
        for (int i = 0; i < n; i++) {-----n+1 times execution
            sum += arr[i];-----n
        }
        System.out.print(sum);-----n
    }
}
```

$sum=5+2n$

From rule 1: remove 5 and 2

Hence the time complexity is $O(n)$

```
public class starpattern {
    public static void main(String[] args) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

Example:

```
for(int i=0;p<=n;i++){
    p=p+i;
}
```

-----0
-----1 0+1
-----2 0+1+2 So on 0+1+2+3+-----+k

$p > n^2$

$k(k+1)/2$

$k^2 + k/2 > n$

$k^2 + k > n$

$k^2 > n$ sobs

$k > \sqrt{n}$

$O(\sqrt{n})$

Options:

1) $O(n)$

2) $O(n^2)$

3) $O(n \log n)$

4) $O(\sqrt{n})$

Example:

```
for(int i=1;i<n;i=i*2)
{
    //statements
}
```

1-----2
2-----4
4-----8
8-----16
16-----32

$2^k \geq n$

$\log 2^k \geq \log n$

$k \log 2 \geq \log n$

$k(1) \geq \log n$

$O(\log n)$

Example

```
for(int i=n;i>=1;i=i/2)
```

```
{-----
```

```
-----}
```

$n, n/2, n/4, n/8, n/16, n/32, \dots, n/2^k$

$n/2^k < 1 \implies n < 2^k$

$\log n < k \log 2$

$O(\log n)$

```
for(int i=0;i<n;i++){
```

```
for(int j=1;j<n;j++){
    }
```

```
}
```

$O(n \log n)$

```
for(int i=1;i<=n;i++){
    for(int j=2;j<=i;j++){
```

```
}
```

```
}
```

$O(n^2)$

```
for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
```

```
}
```

```
}
```

$O(\log(\log n))$

Example:

```
for(int i=0;i*i<n;i++)
```

```
{-----
```

```
-----}
```

1,4,9,16----- n^2

$k^2 < n$

Sobs

$O(\sqrt{n})$

Remove duplicate:

```
public static removeduplicate(int[] num)
```

```
Set<Integer> set=new HashSet<>();
```

```
{
```

```
    for(int num:nums){
```

```
        set.add(num);
```

```
    }
```

```
return set.size();
```

```
}
```

Time complexity: $O(n)$

Space complexity: $O(n)$

RECURSION

Print n natural numbers :

Using loop:

```
import java.util.Scanner;

public class printnaturalnumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a positive integer: ");
        int n = sc.nextInt();
        sc.close();
        System.out.println("Natural numbers from 1 to " + n + ":");
        for (int i = 1; i <= n; i++) {
            System.out.print(i + " ");
        }
    }
}
```

RECUSION:

n==5	5==0(F)
4	4==0(F)
3	3==0(F)
2	2==0(F)
1	1==0(F)
0	TRUE
	If true stop

Public static void main(String[] args){

int number=5;

ravi(number);

}

Static void ravi(int n)

{


```

    if(n==0)    return ;
    ravi(n-1);
    sop(n); }

```

Step over: 1 2 3 4 5

- To print natural numbers in reverse order we need to write highlighted content before the base condition

```

Public static void main(String[] args){
    int number=5;
    ravi(number);
}

```

```

Static void ravi(int n)
{

```

```

    sop(n);
    ravi(n-1);
    if(n==0)    return ;
    sop(n);
}

```

```

public class naturalnumbersreverserecursion {
    public static void main(String[] args) {
        int n = 10;
        printNaturalNumbersReverse(n);
    }
    public static void printNaturalNumbersReverse(int n) {
        if (n == 0) {
            return;
        }
        System.out.print(n + " ");
        printNaturalNumbersReverse(n - 1);
    }
}

```

Sum of natural numbers:

```

import java.util.Scanner;

public class sumofnnaturalnumbersrecursion {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

```

```

        System.out.print("Enter a positive integer: ");
        int n = sc.nextInt();
        sc.close();
        int result = sumOfNaturalNumbers(n);
        System.out.println("Sum of first " + n + " natural numbers is: " + result);
    }

    public static int sumOfNaturalNumbers(int n) {
        if (n == 1) {
            return 1;
        } else {
            return n + sumOfNaturalNumbers(n - 1);
        }
    }
}

```

Factorial of given number:

```

import java.util.Scanner;
public class factorialrecursion {
    int factorial(int n) {
        if (n == 0) {
            return 1;
        } else {
            return n * factorial(n - 1);
        }
    }

    public static void main(String[] args) {
        factorialrecursion fr = new factorialrecursion();
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a positive integer: ");
        int n = sc.nextInt();
        sc.close();
        int result = fr.factorial(n);
        System.out.println("Factorial of " + n + " is: " + result);
    }
}

```

TYPES OF RECURSIONS:

1. **DIRECT RECURSION:** Function calls itself directly
2. **INDIRECT RECURSION:** Function A calls B which calls A

3. **TAIL RECURSION:** recursion call is the last thing executed
4. **NON-TAIL RECURSION:** recursion call is first thing to execute

To find the count of digits of given number:

```
public class countofdigitsrecursion {
    int countDigits(int n) {
        if (n == 0) {
            return 0;
        } else {
            return 1 + countDigits(n / 10);
        }
    }
    public static void main(String[] args) {
        countofdigitsrecursion cdr = new countofdigitsrecursion();
        java.util.Scanner sc = new java.util.Scanner(System.in);
        System.out.print("Enter a positive integer: ");
        int n = sc.nextInt();
        sc.close();
        int result = cdr.countDigits(n);
        System.out.println("Number of digits in " + n + " is: " + result);
    }
}
```

Power of a given number using recursion:

```
public class powerofnumrecursion {

    public static void main(String[] args) {
        int x = 2;
        int n = 5;
        System.out.println("Result: " + power(x, n));
    }

    public static int power(int x, int n) {
        if (n == 0) {
            return 1;
        }
        return x * power(x, n - 1);
    }
}
```

GCD of a,b:

```
public class gcdrecursion {

    public static void main(String[] args) {
        int a = 12;
        int b = 8;
        System.out.println("GCD of " + a + " and " + b + " is: " + gcd(a, b));
    }

    public static int gcd(int a, int b) {
        if (b == 0) {
            return a;
        }
        return gcd(b, a % b);
    }
}
```

Fibonacci series :

Find the n fibonacci number :

```
public class nthfibonaccirecursion {

    public static void main(String[] args) {
        int n = 5;
        System.out.println(fibonacci(n));
    }

    public static int fibonacci(int n) {
        if (n == 1 || n == 0) {
            return n;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Print all numbers of fibonacci series:

```
public class fibonaccirecursion {
    public static void main(String[] args) {
        int n = 10;
        System.out.println("Fibonacci sequence:");
        for (int i = 0; i < n; i++) {
```

```

        System.out.print(fibonacci(i) + " ");
    }
}
public static int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
}

```

Reverse a given number using recursion:

```

public class reversenumberrecursion {
    public static void main(String[] args) {
        int num = 12345;
        System.out.println("Reversed number: " + reverseNumber(num, 0));
    }
    public static int reverseNumber(int num, int reversed) {
        if (num == 0) {
            return reversed;
        }
        return reverseNumber(num / 10, reversed * 10 + num % 10);
    }
}

```

5-1-2026;

Monday;

Declaration of collection frameworks:

Interface<datatype> obj=new className<>();

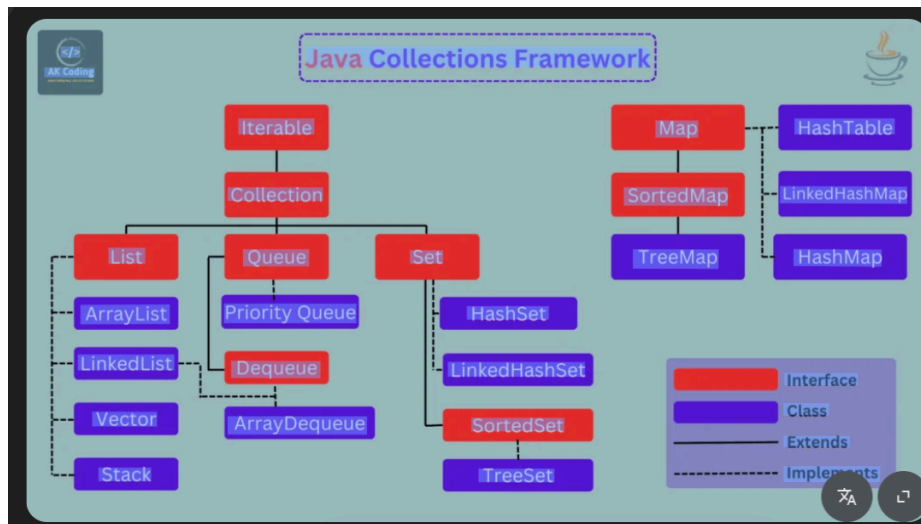
List<String> list =new ArrayList<>(); //list declaration

Set<Integer> set=new HashSet<>(); //set declaration

Queue<String> queue=new LinkedList<>();

- List is a ordered elements and they allow duplicates
- Set is unordered and no duplicates

- Map is unordered ,no duplicates key
- Queue follows FIFO OR LILO



```
List<Integer>list=new ArrayList<>();  
ArrayList<Integer>list1=new ArrayList<>();
```

Name	Base Class	Base Interface	AD	AN	Inserted Order?	Sorted Order?	Synchronized	Random Access	Default Capacity	Description
ArrayList	AbstractList	List	Yes	Yes	Yes	No	No	Yes	10	It supports dynamic arrays that can grow as needed.
LinkedList	Abstract SequentialList	List, Deque, Queue	Yes	Yes	Yes	No	No	Yes	0	It provides a Linked List data structure.
HashSet	AbstractSet	Set	No	Yes	No	No	No	No	16	It creates a collection that uses hash table for storage.
LinkedHashSet	HashSet	-	No	Yes	Yes	No	No	No	16	It creates a Linked List with no duplicate elements.
TreeSet	AbstractSet	Navigable Set	No	No	No	Yes	No	No	16	It creates a collection that uses tree for storage. By default, objects are stored in ascending order.
PriorityQueue	Abstract Queue	Queue	Yes	No	No	Yes	No	No	11	It creates a queue that is prioritized based on queue's comparator.
ArrayDeque	Abstract Collection	Deque	Yes	No	Yes	No	No	No	16	It creates a dynamic array.
EnumSet	AbstractSet	Set	No	No	Yes	No	No			It is specifically for use with elements of enum type.

	Insertion Order	Duplicate Elements	Sorting	Null Elements	Synchronized / Thread Safe
LIST					
Array List	Yes	Yes	No	Yes	No
Linked List	Yes	Yes	No	Yes	No
SET					
Hash Set	No	No	No	Yes	No
Linked Hash Set	Yes	No	No	Yes	No
Tree Set	No	No	Yes (Ascending)	No	No
MAP					
Hash Map	No	Unique Key & Duplicate Values	No	1 Null Key & Multiple Null Values	No
Linked Hash Map	Yes	Unique Key & Duplicate Values	No	1 Null Key & Multiple Null Values	No
Tree Map	No	Unique Key & Duplicate Values	Yes (Ascending Order of Keys)	No Null Key & Multiple Null Values	No

Code:

Class main{

Public static void main(String[] args){

ArrayList<Integer> list=new ArrayList<>();

list.add(10);

list.add(85);

```

list.add(null);
list.add(25);
list.add(36);
list.add(2,4);    //valid
Sop(list.remove(2));
list.clear();
sop(list);
}

```

List methods:

- **add(E e):** adds an element to end of list
- **get(int index):** return element at specified location
- **set(int index):** replace elements at the given index with the given element
- **remove(int index):** removes element of the specified index
- **size():** return number of elements in list
- **clear():** removes all the elements of the list
- **isEmpty():** return true if list is empty
- **contains(Object o):** return true if list contains specified element
- **indexOf(Object o):** return index of first occurrence of specified element in list

Linkedlist methods:

addFirst(E e)/addLast(E e):

getFirst(E e)/getLast(E e): retrieves

removeFirst()/removeLast(): removes element

offerFirst(E e)/offerLast(E e): -----Queue style insert

peekFirst()/peekLast(): -----queue style retrieval

Code:

```

Public class main{
    Public static void main(String[] args){

```



```

List<Integer> list=new LinkedList<>();
list .addFirst(20);
list.addLast(30);
list.addFirst(10);
list.addLast(40);
list.removeFirst();
list.removeLast();
System.out.print(list);
}
}

```

Write a java program to get integer array elements as user input:
ip:

[14,286,3113,6,17]

Op:[2,3,4,1,2]-----count of even digits of count

```

public class counttheevencountofdigits {
    public static void main(String[] args) {
        int[] arr = { 14, 286, 3113, 6, 17 };
        int evenCount = 0;
        for (int num : arr) {
            int digitCount = 0;
            int temp = num;
            while (temp != 0) {
                temp /= 10;
                digitCount++;
            }
            if (digitCount % 2 == 0) {
                evenCount++;
            }
        }
        System.out.println("Count of numbers with even number of digits: " + evenCount);
    }
}

```

Check if element exist in a list

Public class main{

Public static void main(String[] argos){

```

    ArrayList<Integer> list=new ArrayList<>();
    list.add(10);
    list.add(85);
    list.add(null);
    list.add(25);
    list.add(36);
    System.out.print(contains(25));
}
}

```

To find the maximum and minimum elements in list:

```

Public class main{
    Public static void main(String[] args){
        List<Integer> list=new LinkedList<>();
        list .addFirst(20);
        list.addLast(30);
        list.addFirst(10);
        list.addLast(40);
        System.out.print(Collections.max(list));
        System.out.print(Collections.min(list));
        System.out.print(Collections.reverse(list));
    }
}

```

Stack operations:

- **push(E e):** add to top
- **pop():** remove the top element
- **peek():** return top element of stack
- **search(Object o):** position from top
- **empty():** check if stack is empty or not

Set methods:

- **add(E e):** adds elements

- **remove(Object o):** removes element
- **contains(Object o):** checks existence
- **size(),clear(),empty()**--size ops
- **Iterator()**-iteration

```

Class main{
Public static void main(String[] args){
    Set<Integer> list=new LinkedHashSet<>();
    System.out.print(set.isEmpty());
    set.add(40);
    set.add(80);
    set.add(10);
    set.add(20);
    set.add(30);
    set.remove(30);
    System.out.print( set.contains(80));
    System.out.print(set);
}

```

Trees methods:

- **first()/last():** smallest /largest element
- **headSet(E toElement):** elements less than given number
- **tailSet(E fromElement):** elements greater than given element
- **subSet(E from,E to)**--range
- **ceiling(E e)/floor(E e)**-closest $\geq/\leq$
- **Order of $O(\log n)$ for add/search/remove, keeps sorted order**

Write a java program to get integer as user input and get integer array elements from the user to remove duplicates elements in the given array

```

package gradious;
import java.util.*;
public class set {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);

```

```

    int n=sc.nextInt();
    Set<Integer> set=new HashSet<>();
    for(int i=0;i<n;i++) {
        set.add(sc.nextInt());
    }
    System.out.print(set);
    sc.close();
}
}

```

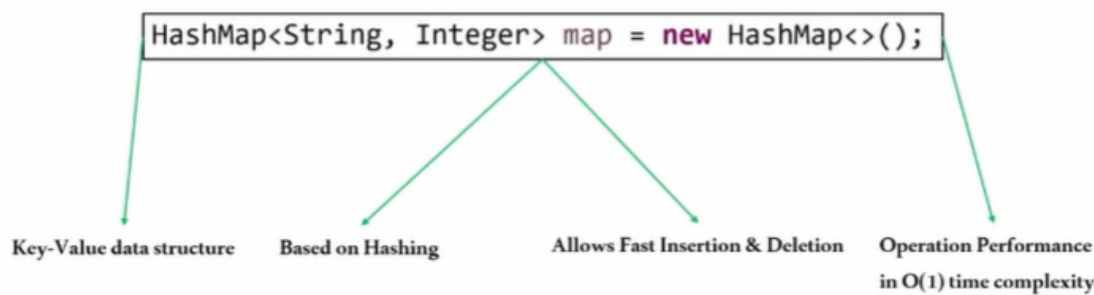
Write a java program to get integer array elements to convert the set into list

```

package gradious;
import java.util.*;
public class set {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        Set<Integer> set=new LinkedHashSet<>();
        for(int i=0;i<n;i++) {
            set.add(sc.nextInt());
        }
        ArrayList<Integer>list= new ArrayList<>(set);
        System.out.print(list);
    }
}

```

What is **HashMap**?



We can use string

---string,integer---integer,integer---string,String---integer,string---character,charcter
 ---integer,Integer---character,charcter---charater

```

HashMap<String,Integer> hm=new HashMap<>();
hm.put("Apple",100);

```

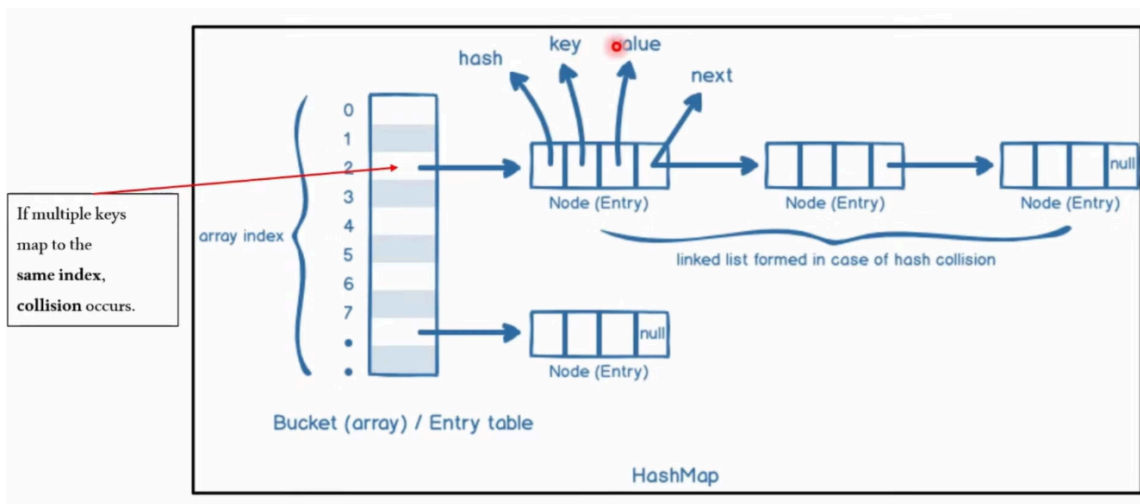
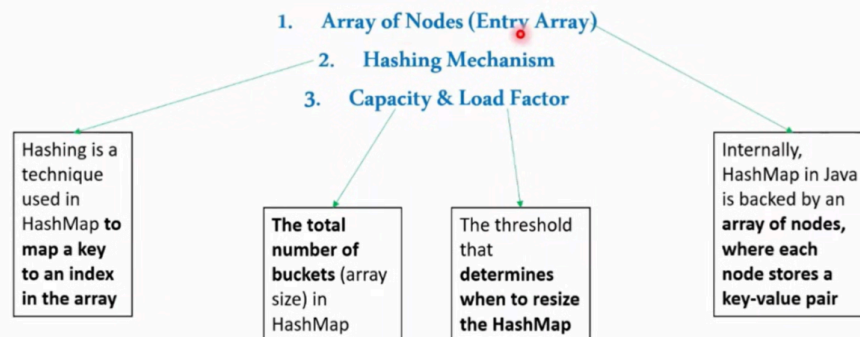
```
hm.put("orange",120);
hm.put("garpes",80);
hm.put("watermelon",50); //in put methods we can add duplicate keys
hm.put("orange",240);
```

```
package gradious;
import java.util.*;
public class hashmap {
    public static void main(String[] args) {
        HashMap<String,Integer> hm=new HashMap<>();
        hm.put("Apple",100);
        hm.put("orange",120);
        hm.put("garpes",80);
        hm.put("watermelon",50); //in put methods we can add duplicate keys
        hm.put("orange",240);
        System.out.println(hm);
    }
}
```

HashMap methods:

- **put(K key,V value):** to add key-value pairs to hashmap
- **get(K key):** retrieve the value associated with the key
- **remove(K key):** remove key value pair associated with the key
- **containsKey(K key):** checks for keys
- **containsValue(V value):** checks for value
- **size():** return size
- **isEmpty():** checks whether empty
- **keySet():** return set of all keys
- **entrySet():** return set of keyvalue pairs
- **values():** returns all values
- Capacity of default hashmap is **16**
- Load factor is **0.75**

Key Components of **HashMap**?



What is Hashing?

Definition:

Hashing is a technique used to store and retrieve data in constant time ($O(1)$) by converting a key into an index using a hash function. Instead of searching element by element, we directly jump to the memory location.

Real-time Analogy

- . Roll number $\rightarrow$ Student details
- . Phone number $\rightarrow$ Contact name

. Username → User profile

You don't scan all data. You compute an index and access directly.

```
import java.util.*;
class Main{
public static void main (String[] args) {
    HashMap<String, Integer> hm = new HashMap
    hm.put("Apple", 100);
    hm.put("Orange", 120);
    hm.put("Grapes", 80);
    hm.put("Watermelon", 50);
    hm.put("Orange", 240);
    int hashCode = "Apple".hashCode();
    int index = hashCode & 16-1;

    System.out.println( hashCode);
    Fyrtem.out.println(index);
}
```

How to find frequency of array elements

```
class Main{
public static void main (String[] args) {
    int[] arr={1, 2, 3, 1, 2, 2, 1, 3};
    int n=arr.length;
    HashMap<Integer, Integer> hm = new HashMap<>();
    for(int num: arr){
        hm.put(num, hm.getOrDefault(num, 0)+1);
    }
    System.out.println(hm);
}
```